You said:

Project: All EKS services helthckeck validation with report

I have to update confluence in jira all issues
Give me all issues
What is the issue simple explanation
Trouble shoot
Validate the issue
Common k8s pod, node, storage, image, and n/w failurs, probe failure etc....

Prepare proper validation with report

ChatGPT said:

Absolutely. For an "**All EKS Services Health Check & Validation**" project, you can maintain one standard Jira/Confluence validation report covering the common EKS failure areas.

Below is a practical structure you can directly use as your project checklist.

EKS Services Health Check & Validation Report

1. Objective

Project: EKS Services Health Check Validation

Objective:
Validate the health and availability of all application services running on Amazon EKS, identify common Kubernetes/infra issues, troubleshoot failures, and document the validation results with evidence.

Validation areas

- Pod health
- Node health
- Deployment / ReplicaSet
- Service
- Ingress / Load Balancer
- ConfigMap / Secret
- Persistent Volume / Storage
- Container image
- ImagePullSecret / ECR
- CPU / Memory
- Liveness / Readiness / Startup probes

- DNS
- Network connectivity
- NetworkPolicy / Security Groups
- HPA
- Cluster Autoscaler
- Events and logs
- Application health endpoints
- Resource limits/requests
- Kubernetes scheduling
- Service-to-service communication

---

2. Overall Health Check

Start with these commands for every EKS cluster.

```
kubectl get nodes
kubectl get pods -A
kubectl get deployments -A
kubectl get services -A
kubectl get ingress -A
kubectl get pvc -A
kubectl get pv
kubectl get events -A --sort-by=.lastTimestamp
```

Check cluster context:

```
kubectl config current-context
kubectl cluster-info
```

Check node resources:

```
kubectl top nodes
kubectl top pods -A
```

Expected result

Validation	Expected
Nodes	Ready
Pods	Running/Completed
Deployments	Available replicas = Desired replicas

Validation	Expected
Services	Correct endpoints
PVC	Bound
Ingress	Correct address
CPU/Memory	Within limits
Events	No abnormal warnings
Probes	Passing
Images	Successfully pulled
Networking	Application communication working

---

3. Pod Failures

Issue

Pod is not running or repeatedly restarting.

Common statuses:

```
Pending
CrashLoopBackOff
Error
ImagePullBackOff
ErrImagePull
OOMKilled
ContainerCreating
Terminating
```

Troubleshooting

```
kubectl get pods -n <namespace>
```

Check detailed information:

```
kubectl describe pod <pod-name> -n <namespace>
```

Check logs:

```
kubectl logs <pod-name> -n <namespace>
```

Previous container logs:

```
kubectl logs <pod-name> -n <namespace> --previous
```

Check events:

```
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

Validate

Confirm:

```
kubectl get pod <pod-name> -n <namespace>
```

Expected:

```
STATUS: Running
READY: 1/1
RESTARTS: 0 or within acceptable range
```

---

4. CrashLoopBackOff

Issue

Container starts but repeatedly crashes.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --previous
```

Check:

- Application startup error
- Wrong environment variable
- Missing Secret
- Missing ConfigMap
- Database connectivity
- Incorrect command/entrypoint
- Application port
- Memory issue
- Dependency failure

Validate

Restart/redeploy after fixing the root cause:

```
kubectl rollout restart deployment <deployment> -n <namespace>
```

Then:

```
kubectl rollout status deployment <deployment> -n <namespace>
kubectl get pods -n <namespace>
```

---

5. Pod Pending

Issue

Pod cannot be scheduled onto a node.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
```

Look at **Events**.

Common reasons:

- Insufficient CPU
- Insufficient memory
- NodeSelector mismatch
- Taints/tolerations
- Affinity rules
- No available nodes
- PVC not available

Check:

```
kubectl get nodes
kubectl describe nodes
kubectl top nodes
```

Validate

Pod should move:

```
Pending → ContainerCreating → Running
```

---

6. Node NotReady

Issue

EKS worker node is unhealthy or unavailable.

Troubleshooting

```
kubectl get nodes
kubectl describe node <node-name>
```

Check:

- CPU
- Memory
- Disk pressure
- Network unavailable
- Kubelet status
- Node conditions

```
kubectl top node <node-name>
```

Check node conditions:

```
kubectl describe node <node-name>
```

Validate

Expected:

```
STATUS: Ready
```

Also ensure workloads are successfully scheduled.

---

7. High CPU / Memory

Issue

Node or pod is consuming excessive resources.

Troubleshooting

```
kubectl top nodes
kubectl top pods -A
```

Check deployment:

```
kubectl describe deployment <deployment> -n <namespace>
```

Check requests/limits:

```
kubectl get pod <pod-name> -o yaml
```

Common causes

- Memory leak
- High traffic
- Incorrect resource limits
- Application issue
- Insufficient node capacity

Validate

CPU and memory should return to an acceptable range after remediation.

---

8. OOMKilled

Issue

Container exceeded its memory limit.

Check:

```
kubectl describe pod <pod-name> -n <namespace>
```

Look for:

```
Reason: OOMKilled
```

Troubleshooting

Check:

```
kubectl top pod <pod-name> -n <namespace>
```

Review:

- Memory usage
- Memory limit
- Application logs
- Traffic level

Validate

After correction:

```
kubectl get pod <pod-name> -n <namespace>
```

Confirm no continuous restarts and no new OOMKilled events.

---

9. ImagePullBackOff / ErrImagePull

Issue

Kubernetes cannot pull the container image.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
```

Check image:

```
kubectl get deployment <deployment> -n <namespace> -o yaml
```

Common reasons:

- Incorrect image name
- Incorrect image tag
- Image doesn't exist
- ECR authentication problem
- IAM permission issue
- Network connectivity
- ImagePullSecret issue

For ECR:

```
aws ecr describe-images \
  --repository-name <repository>
```

Validate

After fixing:

```
kubectl get pods -n <namespace>
```

Expected:

```
Running
```

---

10. Service / Endpoint Failure

Issue

Pod is running but application is not reachable through Kubernetes Service.

Troubleshooting

```
kubectl get svc -n <namespace>
kubectl describe svc <service> -n <namespace>
```

Check endpoints:

```
kubectl get endpoints <service> -n <namespace>
```

Or:

```
kubectl get endpointslices -n <namespace>
```

Check Service selector:

```
kubectl get svc <service> -n <namespace> -o yaml
```

Compare with pod labels:

```
kubectl get pods -n <namespace> --show-labels
```

Common issue

Service selector doesn't match pod labels.

Validate

Service should have healthy endpoints.

---

11. Readiness Probe Failure

Issue

Application is running but Kubernetes doesn't consider it ready.

Example:

```
Readiness probe failed
```

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
```

Check:

- Probe URL
- Port
- Path
- Application startup time
- Network connectivity
- Application health endpoint

Test from inside pod:

```
kubectl exec -it <pod-name> -n <namespace> -- curl localhost:<port>/<health-path>
```

Validate

Expected:

```
READY: 1/1
```

And Service should have endpoints.

---

12. Liveness Probe Failure

Issue

Kubernetes thinks the application is unhealthy and restarts the container.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
```

Check:

- Wrong health endpoint
- Incorrect port
- Application startup time
- Timeout
- Initial delay
- Application dependency failure

Validate

Confirm:

```
kubectl get pods -n <namespace>
```

Restart count should stop increasing.

---

13. Startup Probe Failure

Issue

Application takes longer to start than the configured startup probe allows.

Troubleshooting

Check:

```
kubectl describe pod <pod-name> -n <namespace>
```

Review:

```
initialDelaySeconds
periodSeconds
failureThreshold
timeoutSeconds
```

Validate

Application should successfully start and become:

```
Ready
```

---

14. ConfigMap Issue

Issue

Application is missing or receiving incorrect configuration.

Troubleshooting

```
kubectl get configmap -n <namespace>
kubectl describe configmap <configmap-name> -n <namespace>
```

Check deployment:

```
kubectl describe deployment <deployment> -n <namespace>
```

Verify environment variables:

```
kubectl exec <pod-name> -n <namespace> -- env
```

Validate

Application starts successfully with the expected configuration.

---

15. Secret Issue

Issue

Application cannot access credentials/configuration stored in Kubernetes Secret.

Check:

```
kubectl get secrets -n <namespace>
kubectl describe secret <secret-name> -n <namespace>
```

Check pod:

```
kubectl describe pod <pod-name> -n <namespace>
```

Common causes:

- Secret doesn't exist
- Wrong Secret name
- Incorrect key
- Secret not mounted
- IAM/External Secrets issue

Validate

Application should start and successfully authenticate to the required dependency.

---

16. PVC / Storage Failure

Issue

Pod cannot mount persistent storage.

Check:

```
kubectl get pvc -A
kubectl get pv
```

Detailed:

```
kubectl describe pvc <pvc-name> -n <namespace>
```

Check:

```
kubectl get storageclass
```

Common issues

- PVC Pending
- StorageClass issue
- EBS volume problem
- Availability Zone mismatch
- CSI driver issue
- Volume attachment failure
- Insufficient permissions

Validate

Expected:

```
PVC: Bound
Pod: Running
Volume: Mounted
```

---

17. EBS CSI Driver Failure

Issue

EKS cannot provision or attach an EBS volume.

Check:

```
kubectl get pods -n kube-system | grep ebs
```

Check:

```
kubectl get csidrivers
```

Troubleshoot:

```
kubectl describe pod <pod-name> -n <namespace>
kubectl get events -A --sort-by=.lastTimestamp
```

Validate

PVC should become:

`Bound`

and the application pod should successfully mount the volume.

---

18. DNS Failure

Issue

Pod cannot resolve Kubernetes service or external hostname.

Test:

```
kubectl exec -it <pod-name> -n <namespace> -- nslookup kubernetes.default
```

Test service:

```
kubectl exec -it <pod-name> -n <namespace> -- nslookup <service-name>
```

Check CoreDNS:

```
kubectl get pods -n kube-system | grep coredns
```

Validate

DNS resolution should work from the application pod.

---

19. Network Connectivity Failure

Issue

Pod cannot communicate with another service or external system.

Troubleshooting

From pod:

```
kubectl exec -it <pod-name> -n <namespace> -- curl -v http://<service>:<port>
```

Check:

- Service
- Endpoints
- NetworkPolicy
- Security Groups
- NACL
- Route tables
- DNS
- Application port

Validate

Confirm successful connection and expected HTTP response.

---

20. Ingress / Load Balancer Failure

Issue

Application is working internally but unavailable externally.

Check:

```
kubectl get ingress -A
kubectl describe ingress <ingress-name> -n <namespace>
```

Check service:

```
kubectl get svc -n <namespace>
```

Check AWS Load Balancer/controller:

```
kubectl get pods -n kube-system
```

Validate:

- DNS
- Listener
- Target group
- Health checks
- Security groups
- Backend service
- Ingress rules

Validate

Test:

```
curl -I https://<application-url>
```

Expected:

```
HTTP 200/expected application response
```

---

21. Deployment / Replica Failure

Issue

Deployment doesn't have the expected number of available replicas.

```
kubectl get deployment -A
```

Check:

```
kubectl describe deployment <deployment> -n <namespace>
```

Check rollout:

```
kubectl rollout status deployment <deployment> -n <namespace>
```

Validate

Expected:

```
DESIRED = CURRENT = READY = AVAILABLE
```

---

22. Failed Deployment / Rollout

Issue

New application version is not successfully deployed.

Check:

```
kubectl rollout status deployment <deployment> -n <namespace>
```

History:

```
kubectl rollout history deployment <deployment> -n <namespace>
```

If required, rollback:

```
kubectl rollout undo deployment <deployment> -n <namespace>
```

Validate

```
kubectl get pods -n <namespace>
kubectl rollout status deployment <deployment> -n <namespace>
```

---

23. HPA Failure

Issue

Horizontal Pod Autoscaler isn't scaling as expected.

Check:

```
kubectl get hpa -A
kubectl describe hpa <hpa-name> -n <namespace>
```

Check metrics:

```
kubectl top pods -n <namespace>
```

Common issues:

- Metrics Server
- Incorrect CPU/memory requests
- Wrong target
- Application not generating load

Validate

Generate/test load if appropriate and verify replicas scale according to the configured policy.

---

24. Node Disk Pressure

Issue

Node has insufficient disk space.

Check:

```
kubectl describe node <node-name>
```

Look for:

```
DiskPressure=True
```

Check:

```
kubectl top nodes
```

Investigate:

- Container images
- Container logs
- Temporary files
- Unused images

Validate

Expected:

```
DiskPressure=False
```

---

25. Kubernetes Events

This should be part of **every health check**.

```
kubectl get events -A --sort-by=.lastTimestamp
```

Look for:

```
Failed
Warning
BackOff
FailedMount
FailedScheduling
Unhealthy
FailedAttachVolume
FailedCreatePodSandBox
```

Document the event, root cause and resolution.

---

26. Final Validation

After fixing an issue, don't simply check whether the pod is Running.

Perform **end-to-end validation**:

Infrastructure

```
kubectl get nodes
```

Application

```
kubectl get pods -n <namespace>
```

Deployment

```
kubectl rollout status deployment/<deployment> -n <namespace>
```

Service

```
kubectl get svc -n <namespace>
kubectl get endpoints -n <namespace>
```

Storage

```
kubectl get pvc -n <namespace>
```

Logs

```
kubectl logs <pod-name> -n <namespace>
```

Events

```
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

Application test

```
curl -v http://<service>:<port>/<health-endpoint>
```

External test

```
curl -I https://<application-url>
```

---

27. Jira Issue Format

For **every issue**, use the same format.

Field	Details
Issue	Short issue name
Service	Application/service name
Namespace	Kubernetes namespace

Field	Details
Environment	Dev/QA/UAT/Prod
Severity	Critical/High/Medium/Low
Symptoms	What was observed
Root Cause	Actual reason
Troubleshooting	Commands/actions performed
Resolution	Fix applied
Validation	How it was verified
Status	Resolved/Open
Evidence	Logs/screenshots/command output

Example

Issue: Pod CrashLoopBackOff

Simple explanation:
Application pod was continuously restarting because the container was failing during startup.

Troubleshooting:

```
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
```

Root Cause:
Application was unable to connect to the required dependency due to incorrect configuration.

Resolution:
Corrected the configuration and restarted the deployment.

Validation:

```
kubectl get pods
kubectl rollout status deployment/<deployment>
kubectl get endpoints <service>
```

Result:
Pod is Running, readiness probe is passing, restart count is stable, and application endpoint is responding successfully.

28. Confluence Master Validation Table

You can maintain one master table like this:

#	Validation Area	Issue	Status	Root Cause	Resolution	Validation
1	Pod	CrashLoopBackOff	Pass	App startup error	Configuration fixed	Pod Running
2	Pod	Pending	Pass	Insufficient resources	Capacity adjusted	Pod Scheduled
3	Node	NotReady	Pass	Node issue	Node recovered	Node Ready
4	Image	ImagePullBackOff	Pass	Invalid image/tag	Image corrected	Image pulled
5	Probe	Readiness failure	Pass	Wrong endpoint	Probe corrected	Ready 1/1
6	Probe	Liveness failure	Pass	Health check timeout	Probe tuning	Stable pod
7	Storage	PVC Pending	Pass	Storage provisioning	CSI/Storage fixed	PVC Bound
8	Network	Service unreachable	Pass	Selector/network issue	Corrected	Connectivity OK
9	DNS	DNS failure	Pass	CoreDNS issue	CoreDNS recovered	DNS OK
10	Service	No endpoints	Pass	Selector mismatch	Selector corrected	Endpoints available
11	Ingress	Application unavailable	Pass	LB/Ingress config	Configuration fixed	HTTP 200
12	Resource	OOMKilled	Pass	Memory limit	Resource tuning	No OOM
13	Deployment	Rollout failed	Pass	New version issue	Deployment corrected	Rollout successful
14	HPA	Scaling failure	Pass	Metrics issue	Metrics corrected	Scaling verified
15	Node	DiskPressure	Pass	Disk usage	Cleanup/scaling	DiskPressure false

Recommended project workflow

For your Jira tickets, follow this exact engineering flow:

Detect → Identify failure → Collect evidence → Troubleshoot → Find root cause → Fix → Validate → Document

And for each service, validate:

Node → Pod → Deployment → Container/Image → Probes → Service → Endpoints → DNS → Network → Storage → Ingress/LB → Application health → Logs/Events

This will give you a proper **EKS health-check project report**, rather than just checking whether pods are `Running` .